

Labo 01 — Conteneurs et architecture Docker

Théorie — ce qu'est réellement un conteneur, et qui fait quoi quand vous tapez `docker run`.

Objectifs

- Savoir dire ce qu'est un conteneur **sans** employer le mot « machine virtuelle légère ».
- Comprendre les deux mécanismes du noyau Linux qui rendent les conteneurs possibles.
- Distinguer les quatre acteurs : client, daemon, image, registry.
- Distinguer une **image** d'un **conteneur** — la confusion la plus coûteuse pour un débutant.
- Lire n'importe quelle commande `docker` et en deviner la structure.

1. Le problème que Docker résout

Une application ne tourne jamais toute seule. Une API Spring Boot a besoin d'un JRE dans une version précise, de variables d'environnement, d'un certificat, d'un fuseau horaire, d'un utilisateur système. Un front Angular *buildé* a besoin d'un serveur web configuré d'une certaine manière. Cet ensemble s'appelle l'**environnement d'exécution**.

Historiquement, cet environnement était installé à la main sur un serveur. Deux conséquences bien connues :

1. « **Ça marche sur ma machine** » — la machine du développeur et le serveur de production ne sont jamais identiques.
2. **Les conflits** — deux applications sur le même serveur veulent deux versions différentes de la même bibliothèque, et l'une des deux perd.

La réponse classique était la **machine virtuelle** (VM) : on donne à chaque application son propre système d'exploitation complet. Ça marche, mais on paie un système entier (plusieurs Go de disque, plusieurs secondes à plusieurs minutes de démarrage, de la RAM réservée) pour faire tourner un seul processus applicatif.

Le conteneur est l'autre réponse : **on isole le processus, pas la machine**.

2. Ce qu'est un conteneur

À retenir — Un conteneur est un **processus ordinaire** de votre machine Linux, à qui le noyau ment sur ce qu'il peut voir et sur ce qu'il peut consommer.

Il n'y a pas de système d'exploitation dans un conteneur. Il n'y a pas d'émulation. Si vous lancez un conteneur `nginx`, il existe sur votre hôte un vrai processus `nginx`, visible dans un `ps aux`, exécuté par le **même noyau Linux** que tout le reste. Ce qui change, c'est ce que ce processus perçoit du monde.

Deux mécanismes du noyau Linux font ce travail.

Les namespaces (espaces de noms) — l'isolation de la vue. Le noyau peut donner à un processus une vue partielle et privée de certaines ressources :

Namespace	Ce qu'il isole	Conséquence visible
<code>pid</code>	Les identifiants de processus	Dans le conteneur, votre application est le PID 1 et ne voit aucun autre processus de l'hôte
<code>net</code>	Interfaces réseau, ports, routes	Le conteneur a sa propre IP ; son port 8080 n'est pas le 8080 de l'hôte
<code>mnt</code>	Les points de montage	Le conteneur voit son propre système de fichiers racine <code>/</code>
<code>uts</code>	Le nom d'hôte	<code>hostname</code> renvoie l'identifiant du conteneur
<code>ipc</code>	Les communications inter-processus	Pas de mémoire partagée avec les autres conteneurs
<code>user</code>	Les UID/GID	Un <code>root</code> dans le conteneur peut n'être qu'un utilisateur banal sur l'hôte

Les *cgroups* (control groups) — la limitation des ressources. Les namespaces cachent, les *cgroups* plafonnent : « ce groupe de processus ne dépassera pas 512 Mo de RAM ni 1,5 cœur CPU ». C'est ce qui empêche un conteneur qui part en vrille d'emporter tout le serveur avec lui.

Conteneur ou VM ?

	Machine virtuelle	Conteneur
Isole	Un ordinateur entier (matériel virtualisé)	Un ou plusieurs processus
Noyau	Le sien	Celui de l'hôte, partagé
Démarrage	Secondes à minutes	Millisecondes
Poids typique	Plusieurs Go	Quelques dizaines de Mo
Frontière de sécurité	Forte (hyperviseur)	Plus faible (un seul noyau à compromettre)

Piège — « Le conteneur partage le noyau de l'hôte » a une conséquence directe : un conteneur Linux ne tourne **que** sur un noyau Linux. Sur macOS et Windows, Docker Desktop démarre discrètement une petite VM Linux ; les conteneurs tournent dedans. La légèreté du conteneur n'est donc pleinement vraie que sur un hôte Linux — celui de vos serveurs de production.

3. Image et conteneur

C'est la distinction fondamentale du reste de la formation.

Une **image** est un modèle **en lecture seule** : un système de fichiers figé (le JRE, votre JAR, les bibliothèques) accompagné de métadonnées (quelle commande lancer, quelles variables d'environnement, quel utilisateur, quel port). Une image ne s'exécute pas, ne consomme pas de CPU, ne « tourne » pas. Elle est inerte, et **immuable**.

Un **conteneur** est une *instance en cours d'exécution* d'une image : l'image plus une fine couche d'écriture propre à cette instance, plus un processus vivant.

L'analogie la plus juste est celle de la programmation objet : l'image est la **classe**, le conteneur est l'**objet**. On peut instancier vingt conteneurs de la même image ; ils partagent le même

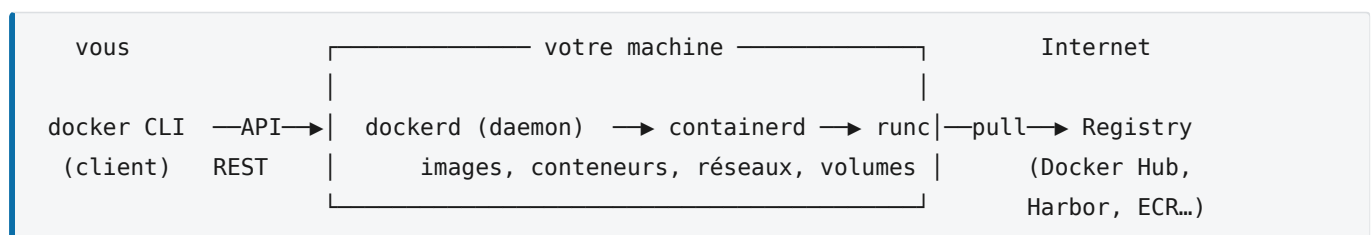
contenu en lecture seule et ont chacun leur couche d'écriture privée.

À retenir — Tout ce que votre application écrit dans un conteneur va dans cette couche d'écriture, qui est **détruite avec le conteneur**. C'est voulu : un conteneur est jetable. La persistance sera l'objet du labo 06.

Autre propriété clé : une image est composée de **couches** (*layers*) empilées, chacune correspondant à une étape de sa construction. Ces couches sont partagées entre images : si dix de vos images partent du même `eclipse-temurin:21-jre`, cette base n'est stockée qu'une fois sur le disque. C'est le sujet du labo 02.

4. L'architecture : qui fait quoi

Docker est une architecture **client / serveur**. Quand vous tapez une commande, quatre acteurs entrent en jeu.



- **Le client `docker`** — le binaire que vous lancez au terminal. Il ne fait presque rien : il traduit votre commande en requête HTTP et affiche la réponse.
- **Le daemon `dockerd`** — le service qui travaille réellement : il construit les images, crée les conteneurs, gère réseaux et volumes. Il écoute par défaut sur une *socket* Unix, `/var/run/docker.sock`.
- **Le registry** — le dépôt distant d'images. Docker Hub est le registry public par défaut ; en entreprise, on utilise presque toujours un registry privé (Harbor, Nexus, GitLab Registry, AWS ECR, Azure ACR).
- **Le runtime** — sous `dockerd`, `containerd` puis `runc` font le travail bas niveau d'appel au noyau. Vous ne les manipulez pas, mais leurs noms apparaissent dans les logs et les offres d'emploi.

Deux conséquences pratiques à comprendre tout de suite :

1. **Le travail se fait côté daemon.** Un chemin monté avec `-v /data:/data` est résolu sur le disque du **daemon**, pas sur le vôtre. Lors d'un `docker build`, c'est l'inverse : le client empaquette le dossier courant (le *build context*) et l'envoie au daemon, qui construit. Sur une machine locale la distinction est invisible ; contre un daemon distant, elle explique la moitié des surprises.
2. **L'accès à `/var/run/docker.sock` équivaut à un accès `root` sur l'hôte.** Qui peut parler au daemon peut lancer un conteneur privilégié et prendre la machine. C'est pour cela que l'appartenance au groupe `docker` n'est pas anodine.

5. Anatomie d'une commande Docker

La CLI moderne suit une grammaire régulière :

```
docker [objet] [action] [options] [cible] [arguments]
```

```
docker container run -d --name api -p 8080:8080 eclipse-temurin:21-jre java -version
#      └─objet─┘ └─action─┘ └─── options ───┘ └── image ─┘ └─ commande ─┘
```

Les objets principaux sont `image`, `container`, `volume`, `network`, `system`. Pour les opérations les plus fréquentes, il existe des **raccourcis historiques** où l'objet est implicite — ce sont eux qu'on lit partout :

Forme complète	Raccourci courant
<code>docker container run</code>	<code>docker run</code>
<code>docker container ls</code>	<code>docker ps</code>
<code>docker image ls</code>	<code>docker images</code>
<code>docker image pull</code>	<code>docker pull</code>
<code>docker container rm</code>	<code>docker rm</code>
<code>docker image rm</code>	<code>docker rmi</code>

Trois commandes de diagnostic à connaître dès maintenant :

```
docker version  # versions du client ET du daemon, séparément
docker info     # état du daemon : nombre de conteneurs/images, stockage, cgroups
docker inspect  # toutes les métadonnées d'un objet, en JSON
```

Piège — `docker version` affiche deux blocs, *Client* et *Server*. Si le second est absent ou remplacé par une erreur de connexion, le daemon ne tourne pas ou vous n'avez pas le droit de lui parler : le problème n'est jamais dans votre commande.

6. En entreprise

Sur une stack Spring Boot + Angular + PostgreSQL, cette théorie se traduit ainsi :

- Le back Spring Boot devient **une image** contenant un JRE et le JAR. La même image, au **digest** près, part sur l'intégration, la recette et la production : c'est la fin du « ça marche sur ma machine ».
- Le front Angular n'est pas conteneurisé « tel quel » : on le *builde* (`ng build`) puis on embarque le résultat statique dans une image nginx. Node ne survit pas à la production — voir le labo 05.
- PostgreSQL est tiré d'une image publique officielle ; on n'écrit pas d'image de base de données, on la configure.
- Ces trois images vivent dans un **registry privé** interne, poussées par la CI.

À retenir

- Un conteneur est un **processus isolé** par les *namespaces* et limité par les *cgroups*, exécuté par le **noyau de l'hôte** — pas une mini-VM.
- Une **image** est un modèle immuable en lecture seule ; un **conteneur** en est une instance vivante avec une couche d'écriture jetable.

- Le client `docker` ne fait qu'envoyer des requêtes au daemon `dockerd`, qui effectue tout le travail : les chemins et le contexte sont ceux du **daemon**.
- Un **registry** stocke et distribue les images ; Docker Hub par défaut, un registry privé en entreprise.
- Les conteneurs sont **jetable**s : tout état non externalisé disparaît à leur suppression.
- L'isolation d'un conteneur est plus faible que celle d'une VM — un seul noyau est partagé. Accès au daemon = accès root.
- La CLI suit `docker <objet> <action>` ; les formes courtes (`docker ps`, `docker run`) sont des raccourcis de cette grammaire.

Vocabulaire

image : modèle immuable, empilement de couches en lecture seule. — **conteneur** : instance en exécution d'une image. — **layer** (couche) : fragment de système de fichiers issu d'une étape de construction. — **registry** : serveur qui stocke et distribue les images. — **repository** : ensemble des versions d'une même image (`postgres`). — **tag** : étiquette d'une version (`postgres:16-alpine`). — **daemon** : le service `dockerd`. — **namespace** : isolation de la vue d'une ressource par le noyau. — **cgroup** : limitation de la consommation de ressources. — **runtime** : `containerd` / `runc`, les composants qui créent réellement le conteneur.